Math Practice App Development Report

Gavin | CSE4095 | Individual Project | October 22, 2026

Repository: https://github.com/Hab1yer/math-practice-app

Application: https://math-practice-app-pi.vercel.app/

Development approach

I used Codex as the AI-assisted development environment for a React and TypeScript math practice application for children ages 7-12. The application supports four arithmetic operations, four difficulty levels, interactive question cards, session summaries, local progress storage, and personalized mistake review. Version 1.0.0 is publicly deployed on Vercel and does not require an account.

I directed development through the supplied specification and phased TODO list. The early work followed one phase at a time: project setup, navigation, configuration, question generation, answer choices, cards, and sessions. The agent maintained implementation notes, a testing record, and meaningful Git commits as features became available. Later, I requested Phases 13-16 together and explicitly postponed their tests. The final release pass executed that deferred coverage before publication. This adjustment changed the order of verification without removing it.

I also requested repository cleanup. Application code, dependencies, and tool configuration were moved into app/, while requirements and development records stayed in docs/. The README explains installation, local execution, production builds, and the hosting directory. This organization made the repository easier to navigate and required the deployment root to point to app/.

The architecture separates the interface from arithmetic generation, session management, mistake review, statistics, and storage. React Context and hooks coordinate application state; React Router handles navigation. Browser localStorage holds preferences, attempts, sessions, and mistake patterns. Statistics and mastery derive from recorded attempts. This avoids needing a backend while keeping progress on the current browser and device.

Prompting strategy

My initial request was "Read handoff.md and continue from the immediate task." It directed the agent to recover the requirements and current scope before editing. Repeated "proceed" prompts then continued the next phase identified in the preceding response. This depended on the checklist and documentation retaining enough context to keep the work ordered.

Prompting strategy continued

A more targeted request was "test the live browser verificatiton again and do an adversarial test". This followed a blocked browser verification attempt. It led to a successful browser retry and tests for duplicate, stale, malformed, and conflicting submissions. The live check double-clicked a result button and confirmed that exactly one attempt was counted. This prompt was useful because it asked the agent to challenge the implementation through observable behavior.

Another effective request was "Can we continue phases 13-16 please and leave the testing after". It identified a specific group of features and a workflow constraint. The agent implemented similar-question generation, mistake review, advanced statistics, and accessibility changes, while documenting behavioral tests as deferred. My later request to finish the remaining phases included completing those tests. The record distinguishes implementation from verified completion.

Debugging experience

One release problem involved automatic completion of a mistake-review session. The expected behavior was to move directly to the summary when the final active mistake became mastered. Testing with the production router exposed a navigation guard reading stale active-session state and blocking that transition. A related issue let a summary redirect compete with an explicit End and leave action.

The agent corrected the guard to consult current session state and moved summary navigation to the result/completion event. Regression tests used the same router API as production. In the subsequent browser flow, three correct linked review answers mastered the saved mistake and ended review automatically without an unwanted leave warning. Statistics then showed eight attempts, seven correct, one incorrect, and 88% accuracy. This connected the fix to both navigation behavior and persisted results.

An earlier validation bug showed why tests must include malformed data. JavaScript array checks were skipping holes in a sparse answer-choice array. A question could therefore appear to have four array slots while containing only three values. A regression test failed before the fix; the validator was changed to expose missing entries as undefined and reject them. The full check then passed. Both examples demonstrate the value of reproducing an issue and checking its result after a change.

Testing and deployment

The final recorded check passed 335 automated tests across 32 files, together with lint, formatting, strict TypeScript checks, and the production build. Generator coverage included 8,000 reproducible operation-and-difficulty samples, boundary values, whole-number division, nonnegative subtraction, and four unique choices. Other tests covered self-reported scoring, duplicate protection, session completion, persistence, corrupted storage, review priority, mastery, settings, and reset failures.

Agent-operated browser checks exercised complete practice and review flows, keyboard selection, saved-card restoration, navigation cancellation, summaries, and statistics. Phone and tablet viewports from 320 to 1024 pixels wide were checked for horizontal overflow. Seven automated axe cases checked structural accessibility. A synthetic history containing 5,000 attempts retained its records and totals, with a final measured storage round trip of 78 ms in the test environment.

Vercel deploys the GitHub main branch with app/ as the project root. A routing fallback allows direct visits and refreshes on application routes. Hosted checks confirmed the public home page, setup, settings, statistics, privacy, missing-session redirects, unknown-route recovery, and preference persistence. The captured production console had no errors or warnings. No login, API key, or local development server is required to use the deployed app.

These checks were performed by the agent; they are not a claim that I personally repeated every test. Actual Safari, Firefox, Edge, screen-reader, 200% zoom, larger system fonts, physical-device, and full browser-process restart checks remain open. Fresh-document and completed-state refresh persistence were verified, but neither substitutes for quitting and reopening the browser process. The testing and release documents preserve these limits.

Reflection

The development record shows that AI can accelerate scaffolding, arithmetic logic, test creation, documentation, and deployment preparation. It can also investigate failures across several layers of an application. The strongest results came when a request named the scope or asked for concrete verification, because the resulting work could be compared with requirements and observed behavior.

The limitations were equally visible: passing service tests did not guarantee correct production-router behavior, and browser viewport checks could not prove physical-device usability. Some test failures came from incorrect test assumptions rather than application defects. AI-generated claims therefore needed evidence and clear boundaries. For this project, effective supervision meant controlling scope, retaining meaningful commits, revisiting deferred tests, and distinguishing implemented features from verified behavior. Understanding the architecture and demonstrating the application remain my responsibilities as the student.